Relación 02 · Sesiones, cookies y PDO

UD 02 · Desarrollo Web en Entorno Servidor · PHP 8.1+

Esta relación contiene seis ejercicios independientes. Los cuatro primeros utilizan PHP sin MySQL; los dos últimos requieren PDO y MySQL. Los archivos de `base/` contienen datos y estructuras mínimas, pero no las soluciones.

Normas comunes

- Para modificar datos, usa `POST`. Los procesadores de acciones deben rechazar `GET` mediante HTTP 405.
- Valida todos los datos en el servidor. Escapa las variables insertadas en HTML con `htmlspecialchars()`.
- Tras modificar datos correctamente, redirige al listado (POST/Redirect/GET).
- Los repositorios de PDO no imprimen ni consultan `$_POST` o `$_GET`: reciben parámetros y devuelven resultados.
- Nunca muestres al usuario errores técnicos con información SQL o contraseñas.

Ejercicio 1 · Control de acceso al panel de incidencias

Objetivo. Autenticar usuarios, conservar su identidad en una sesión, comprobar permisos según su perfil y cerrar el acceso correctamente.

Material inicial

En `base/usuarios.php` se proporcionan dos cuentas con contraseñas almacenadas mediante `password_hash()`. `base/incidencias.php` contiene cinco incidencias identificadas por un número. Completa `login.php`, `panel.php`, `revisar.php`, `informe.php` y `logout.php`. No necesitas base de datos.

Cuentas para las pruebas:

- **alba@centro.test** / **AccesoAlba26!** — perfil `coordinacion`.
- **marcos@centro.test** / **AccesoMarcos26!** — perfil `tecnico`.

Requisitos

1. `login.php` presenta un formulario de **correo** y **contraseña**. Al enviarlo mediante `POST`, comprueba que ambos campos existen, busca el correo sin distinguir mayúsculas y minúsculas y valida la contraseña con `password_verify()`. Si falla, muestra **«Datos de acceso incorrectos.»** y mantiene el formulario.
2. Si las credenciales son válidas, regenera el identificador mediante `session_regenerate_id(true)` y guarda en `$_SESSION` **nombre, correo y perfil**. Redirige a `panel.php`. Si una persona ya identificada abre `login.php`, redirige directamente al panel.
3. `panel.php` solo puede mostrarse cuando la sesión contiene un usuario identificado. Enseña un saludo con su nombre y perfil, las **cinco incidencias** del archivo de datos y su estado **Pendiente**/**Revisada**. Si no hay sesión, redirige al acceso.
4. Cada incidencia pendiente tiene un botón **Marcar como revisada**, que envía su identificador por `POST` a `revisar.php`. Conserva en `$_SESSION['revisadas']` los IDs de las incidencias revisadas por esa persona. Repetir un ID no lo duplica; un ID desconocido se rechaza sin modificar la sesión. El estado no debe guardarse en cookies ni en los datos originales.
5. `informe.php` es una página privada **exclusiva de `coordinacion`**. Muestra el número de incidencias pendientes y revisadas de la sesión actual. Una persona identificada con perfil `tecnico` que intente acceder directamente recibe **HTTP 403**, sin ver el informe.
6. `logout.php` acepta únicamente `POST`, vacía `$_SESSION`, solicita la eliminación de la cookie de sesión con sus parámetros, destruye la sesión y redirige a `login.php`. El panel incluye un formulario para salir; no debe bastar con visitar `logout.php` mediante `GET`.

7. Coloca `session_start()` al principio de los scripts que necesiten acceder a la sesión. Escapa los valores mostrados en HTML con `htmlspecialchars()` y no envíes HTML antes de las redirecciones.

Pruebas

- Una contraseña incorrecta no permite entrar y muestra un mensaje genérico.
- Marcos puede revisar una incidencia, recargar el panel y seguir viéndola como revisada; no puede abrir `informe.php`.
- Alba puede entrar al informe y el recuento cambia al marcar incidencias.
- Modificar manualmente el ID enviado a `revisar.php` por uno inexistente no altera el listado.
- Tras cerrar sesión, una petición directa a `panel.php` vuelve al formulario de acceso.

Ejercicio 2 · Registro de turnos con caducidad de sesión

Objetivo. Controlar un inicio de sesión con intentos fallidos, caducidad por inactividad y acciones privadas que dependen del estado guardado en la sesión.

Material inicial

`base/cuentas.php` contiene dos cuentas de prueba con hashes de contraseña. Implementa `acceso.php`, `panel.php`, `registrar.php`, `sesion.php` y `salir.php`. No utilices MySQL ni almacenamiento permanente: los movimientos se conservan exclusivamente durante la sesión iniciada.

Cuentas para las pruebas:

- **ana@turnos.test** / **EntradaAna26!**.
- **jose@turnos.test** / **EntradaJose26!**.

Requisitos

1. `acceso.php` permite identificarse con correo y contraseña. Usa `password_verify()` y, si el acceso es correcto, ejecuta `session_regenerate_id(true)`, guarda el **correo** y el **nombre** en `$_SESSION` e inicializa una lista vacía de movimientos y la hora de última actividad con `time()`.
2. Lleva en la sesión el contador `intentos` de autenticaciones fallidas. Cada `POST` que incluya ambos campos, pero tenga credenciales incorrectas, suma un intento. Al alcanzar **tres fallos consecutivos**, fija `bloqueado_hasta` a `time() + 90`. Durante esos **90 segundos**, el acceso se rechaza incluso con la contraseña correcta y se muestra cuánto falta aproximadamente. Al finalizar el bloqueo, permite probar de nuevo con cero intentos. Un acceso correcto también reinicia ambos datos.
3. Para cualquier página privada, `sesion.php` debe proporcionar una comprobación reutilizable. Si no hay usuario identificado, redirige a `acceso.php`. Si han transcurrido **más de 180 segundos** desde la última actividad, finaliza la sesión y redirige a `acceso.php?motivo=expirada`. En peticiones válidas actualiza la última actividad. La caducidad se comprueba **en el servidor**, no con JavaScript.
4. `panel.php` muestra el nombre y una tabla de movimientos de la sesión: tipo (**Entrada**/**Salida**) y fecha y hora generadas por el servidor. Incluye dos formularios que envían `POST` a `registrar.php`, con el campo `tipo` igual a `entrada` o `salida`.
5. `registrar.php` solo admite `POST` y necesita una sesión autenticada y vigente. La **primera operación debe ser Entrada**. Después, los tipos deben **alternarse**: no se permiten dos entradas seguidas ni dos salidas seguidas. Si una operación es inválida, no cambia la lista; si es válida, añade un movimiento con la hora actual y redirige al panel.
6. `salir.php` acepta solo `POST` y realiza el cierre completo de sesión, incluida la eliminación de su cookie; después redirige a `acceso.php`. El cierre voluntario y el vencimiento de la sesión deben impedir volver a consultar los movimientos privados.
7. No incluyas la contraseña ni los hashes en `$_SESSION`. Valida los valores de `POST` en el servidor y usa `htmlspecialchars()` al insertar datos en HTML.

Pruebas

- Dos contraseñas incorrectas y una correcta permiten entrar y reinician el contador.
- Tres errores seguidos impiden acceder durante 90 segundos incluso con la clave correcta.
- Una petición directa a `registrar.php` sin sesión no añade movimientos.
- Se acepta **Entrada → Salida → Entrada**; se rechaza **Entrada → Entrada**.
- Con la sesión inactiva más de 180 segundos, la siguiente petición privada redirige al acceso y no conserva los movimientos.

Ejercicio 3 · Cuestionario por etapas con sesiones

Objetivo. Mantener el estado de una actividad que avanza por etapas, evitando que se responda varias veces a una misma pregunta.

Material inicial

En `base/preguntas.php` se proporcionan cuatro preguntas de opción múltiple y la respuesta correcta. Crea el funcionamiento de `index.php`, `responder.php` y `reiniciar.php` sin modificar las preguntas.

Requisitos

1. Al entrar sin una partida activa, muestra un formulario que solicita un **alias** de 2 a 30 caracteres y el botón **Comenzar**. El formulario envía `POST` a `index.php`.
2. Valida el alias y guarda en `$_SESSION['cuestionario']` la información necesaria: alias, índice de pregunta actual y aciertos. Redirige a `index.php`.
3. Presenta **solo la pregunta actual** y sus opciones mediante botones de radio. El formulario envía `POST` a `responder.php`. No envíes al navegador la respuesta correcta ni confíes en un índice de pregunta recibido del cliente.
4. En `responder.php`, comprueba que la sesión contiene una partida sin terminar. Acepta exclusivamente una opción válida de la pregunta actual. Si la opción no existe o falta, no avances ni sumes puntos y muestra un error comprensible. Una respuesta válida pero equivocada sí hace avanzar, aunque no suma aciertos.
5. Por cada respuesta válida, avanza **una pregunta** y suma un acierto únicamente cuando corresponda. Después de la cuarta respuesta muestra el alias y la nota sobre cuatro. No debe poder responderse de nuevo.
6. El botón **Reiniciar** debe enviar `POST` a `reiniciar.php`, que elimina solo `$_SESSION['cuestionario']` y redirige a `index.php`.
7. Recargar `index.php` nunca avanza preguntas ni suma aciertos. Acceder directamente a `responder.php` sin partida redirige a `index.php`.

Pruebas

- Un alias de una letra se rechaza y uno de tres letras comienza el cuestionario.
- Recargar la primera pregunta mantiene la puntuación y la posición.
- Enviar manualmente una opción `99` no avanza ni puntúa.
- Tras cuatro respuestas se muestra el resultado y no hay botones para responder.
- Reiniciar vuelve al formulario de alias.

Ejercicio 4 · Tres rutas favoritas guardadas en cookies

Objetivo. Gestionar una lista ordenada de favoritos persistida en una cookie JSON. No hay carrito, cantidades ni precios.

Material inicial

En `base/rutas.php` hay cinco rutas con código, nombre, kilómetros y dificultad. Implementa `index.php`, `accion.php` y `vaciar.php`. No uses base de datos ni sesiones.

Requisitos

1. `index.php` muestra las cinco rutas. Cada una tiene un botón **Añadir** o **Quitar** según se encuentre entre las favoritas. Estos botones envían `POST` a `accion.php` con los campos `accion` (`anadir` o `quitar`) y `codigo`.
2. Guarda las favoritas en **una sola cookie** llamada `rutas_favoritas`, con un JSON como `["R1","R3"]`, duración de **30 días**, ruta `/`, `httponly=true` y `samesite=Lax`.
3. Se permiten **como máximo tres** rutas, **sin duplicados**, y se conserva el orden en el que se han añadido. Añadir una ruta ya seleccionada o quitar una que no existe no debe alterar las restantes.
4. Si el usuario intenta añadir la cuarta, no cambies las favoritas y muestra el aviso **«Solo puedes guardar tres rutas.»**. Puedes redirigir a `index.php?aviso=limite`.
5. Si la cookie no existe, no contiene JSON válido, o no contiene un array, comienza con favoritos vacíos. Si hay códigos desconocidos o repetidos, ignóralos y conserva, por orden, los **tres primeros códigos válidos distintos**.
6. Muestra debajo del catálogo el listado ordenado de favoritas y la **suma de kilómetros**, calculada **exclusivamente con los datos de `rutas.php`**, nunca con datos de la cookie. Si la lista está vacía, muestra **«No tienes rutas favoritas.»**.
7. **Vaciar favoritos** envía `POST` a `vaciar.php`, que caduca la cookie y redirige. `accion.php` y `vaciar.php` responden con `405` ante `GET`.

Pruebas

- Añadir R1 y R3 mantiene ese orden y suma sus kilómetros.
- Añadir R1 otra vez no la duplica. Quitar R1 conserva R3.
- Una cuarta ruta no se incorpora y aparece el aviso.
- Si la cookie contiene `["R3","X99","R3","R1"]`, solo aparecen R3 y R1.
- Vaciar favoritas deja la lista vacía tras redirigir.

Ejercicio 5 · Préstamos de equipos con PDO

Objetivo. Programar operaciones con dos tablas relacionadas, `JOIN`, cálculos de disponibilidad y una transacción que evite prestar unidades inexistentes.

Preparación

Ejecuta `base/schema.sql`: crea `ud2_prestamos`, las tablas `equipo(id,nombre,unidades)` y `prestamo(id,equipo_id,persona,fecha_prestamo,fecha_devolucion)` y tres equipos de muestra. Configura PDO en `config.php` e implementa `EquipoRepositorio.php` e `index.php`.

La disponibilidad no se almacena: es `equipo.unidades` menos los préstamos que tienen `fecha_devolucion IS NULL`.

Métodos requeridos

Método	Operación y devolución
`listarEquipos(): array`	Devuelve `id`, `nombre`, `unidades` y `disponibles` de cada equipo, ordenados por nombre. Calcula disponibilidad en SQL sin ejecutar una consulta por cada equipo.

Método	Operación y devolución
`listarPrestamos(): array`	Devuelve los préstamos con nombre del equipo (`JOIN`), poniendo primero los abiertos y luego los cerrados.
`prestar(int $equipoId, string $persona): int`	Crea un préstamo abierto y devuelve su ID si hay disponibilidad; de lo contrario, lanza `DomainException`.
`devolver(int $prestamoId): bool`	Registra `CURDATE()` **solo en préstamos abiertos**. Devuelve si se modificó una fila.

Requisitos

1. Muestra una tabla **Equipo / Unidades / Disponibles** y un formulario para prestar: `equipo_id` y `persona` (2 a 80 caracteres sin espacios extremos). Valida IDs positivos, nombre y existencia del equipo.
2. Muestra otra tabla de préstamos: ID, equipo, persona, fecha de préstamo y fecha de devolución. Los abiertos tienen un botón **Devolver** que envía `POST`; los cerrados no.
3. Al prestar, comienza una transacción, ejecuta `SELECT ... FOR UPDATE` sobre el equipo para bloquearlo, calcula cuántos préstamos siguen abiertos, comprueba la disponibilidad e inserta el préstamo con `CURDATE()` y fecha de devolución `NULL`. Usa `commit()` y `rollBack()` según corresponda.
4. Si no quedan unidades, muestra **«No quedan unidades disponibles.»** sin insertar nada. Si se intenta devolver dos veces, conserva la primera fecha de devolución.
5. Las operaciones válidas redirigen a `index.php`. No imprimas resultados dentro del repositorio ni expongas errores técnicos de PDO.

Pruebas

- Un equipo con una unidad admite un préstamo, pero no dos simultáneos.
- Al devolverlo, vuelve a quedar disponible una unidad.
- Una segunda devolución no cambia la fecha ya guardada.
- Los equipos con dos unidades permiten dos préstamos, pero no tres.

Ejercicio 6 · Talleres con lista de espera automática

Objetivo. Implementar una regla de negocio con aforo limitado, inscripciones duplicadas y promoción automática de personas en espera al cancelar.

Preparación

Ejecuta `base/schema.sql`: crea `ud2_talleres`, la tabla `taller(id,titulo,plazas)` y la tabla `inscripcion(id,taller_id,nombre,email,estado,creado)`. `estado` puede ser `confirmada` o `espera`; la pareja `(taller_id,email)` es única. Hay dos talleres precargados: uno con dos plazas y otro con tres.

Métodos requeridos

Método	Operación y devolución
`listarTalleres(): array`	Devuelve cada taller con `plazas`, `confirmadas` y `en_espera`, mediante SQL y ordenados por título.
`listarInscripciones(): array`	Devuelve inscripciones junto al título del taller (`JOIN`), ordenadas por taller, confirmadas antes que espera e ID ascendente.

Método	Operación y devolución
`inscribir(int $tallerId, string $nombre, string $email): string`	Inscribe y devuelve exactamente `confirmada` o `espera`.
`cancelar(int $inscripcionId): bool`	Cancela y, si estaba confirmada, promociona a la primera persona en espera del mismo taller. Devuelve `false` si no existe.

Requisitos

1. Muestra por taller las plazas totales, confirmadas y personas en espera. Añade un formulario con taller, nombre (2 a 80 caracteres) y correo válido. Normaliza el correo a minúsculas y sin espacios extremos.
2. Si hay plazas disponibles, inscribe con estado `confirmada`. Si el aforo está completo, guarda con estado `espera`. **No se rechaza** una inscripción solo por estar lleno.
3. Impide dos inscripciones del mismo correo en un taller, incluso si la primera está en espera. El correo sí puede inscribirse en otro taller distinto.
4. Muestra un listado con nombre, correo, taller, estado y botón **Cancelar** por `POST`.
5. Al cancelar una persona confirmada, asciende a `confirmada` la persona en espera del mismo taller con **menor ID**. Cancelar a una persona en espera no asciende a nadie.
6. `inscribir()` y `cancelar()` deben utilizar transacciones. Bloquea la fila del taller con `SELECT ... FOR UPDATE` antes de tomar decisiones sobre aforo. Para cancelar, identifica el taller, bloquéalo y vuelve a verificar la inscripción. El borrado y la posible promoción son **una sola operación atómica**.
7. Utiliza la lógica transaccional para que nunca haya más personas confirmadas que plazas. La restricción `UNIQUE` de MySQL impide los duplicados por taller y correo. Evita mostrar mensajes SQL literales y redirige tras cada cambio válido.

Pruebas

- En el taller de dos plazas, las dos primeras personas quedan confirmadas; la tercera y cuarta, en espera.
- Si cancelas una de las dos confirmadas, asciende la **tercera**, no la cuarta.
- Cancelar a la cuarta (en espera) no cambia las confirmadas.
- El mismo correo no puede apuntarse dos veces al mismo taller, pero sí a otro taller.
- Sin lista de espera, cancelar una confirmada deja una plaza libre para el siguiente registro.